Post-Generation Hallucination Detection Middleware for Large Language Models

System Design, Datasets, and Methods

Yash Deshpande

September 2026

Abstract

Generative AI systems can produce fluent and convincing responses that contain incorrect, unsupported, or fabricated information, commonly referred to as hallucinations. This project proposes a post-generation hallucination detection middleware that operates between a large language model (LLM) API and the end user, without modifying or retraining the underlying model. The system intercepts generated responses, decomposes them into individual factual claims, and retrieves relevant evidence from configurable trusted knowledge sources using vector embeddings, retrieval-augmented generation (RAG), and external search services. Each claim is then evaluated against the retrieved evidence using semantic verification and natural language inference (NLI), classifying it as supported, contradicted, or unsupported. A risk-scoring layer aggregates verification signals into claim-level and response-level hallucination scores, while an explanation layer surfaces the evidence and reasoning behind flagged claims. The system returns an enriched API response containing the original output together with verification metadata, evidence, risk scores, and explanations for integration into downstream applications. The architecture is model-agnostic and configurable: different LLM providers, knowledge bases, vector databases, and search tools can be plugged in without changing the pipeline. We describe the system design in detail, including block diagrams and pseudocode for each pipeline stage, and ground the evaluation plan in current benchmarks - RAGTruth, AVeriTeC, FActScore, LLM-AggreFact (MiniCheck), and RAGChecker - with FEVER and HaluEval retained as baselines for comparability with prior work. A background section explains how LLMs generate text through autoregressive next-token prediction, why that process differs structurally from human writing, and why those differences make an external verification layer necessary.

1. Introduction

Large language models now sit behind search assistants, customer support bots, and document drafting tools, but they share a well-documented failure mode: they produce text that is coherent, confident, and wrong [4]. Because the failure is fluent, end users cannot reliably tell a correct answer from a fabricated one, and downstream applications built on LLM APIs inherit that risk.

Most existing mitigations intervene inside the model (fine-tuning, RLHF, retrieval during generation) and therefore require control over the model itself. Many production systems, however, call third-party LLM APIs where no such control exists. This project takes the complementary approach: verify after generation, from the outside. The middleware treats the LLM

as a black box, intercepts each response, and checks every factual claim it contains against trusted evidence before the response reaches the user.

This document covers four things. Section 2 explains how LLMs actually generate text and how that process differs from human writing, since those differences are the reason post-generation verification is needed at all. Section 3 presents the datasets and benchmarks chosen for development and evaluation, including why the initially planned FEVER and HaluEval are not sufficient on their own. Section 4 describes the full system architecture with block diagrams and pseudocode. Section 5 covers the knowledge base design, including why a model pretraining corpus (such as the IBM Granite training mixture) is the wrong tool for verification and what to use instead.

2. How LLMs Generate Text, and Why It Differs from Human Writing

2.1 Autoregressive next-token prediction

A modern LLM does not write sentences the way a person does. It is a next-token predictor. Input text is first split into tokens (subword units, roughly three quarters of a word each in English), and the model - almost always a Transformer, the architecture introduced by Vaswani et al. [1] - computes a probability distribution over its entire vocabulary for the single next token, conditioned on every token that came before it:

```
P(x_t | x_1, x_2, ..., x_{t-1})
```

Generation then repeats a simple loop: sample one token from that distribution, append it to the sequence, feed the longer sequence back in, and sample again, until an end-of-text token appears or a length limit is hit. A full paragraph is the product of hundreds of these independent local decisions. There is no outline, no plan for where the answer is going, and no stage where the model checks what it has written against a source. Each token is chosen because it is a statistically plausible continuation of the tokens so far, nothing more.

2.2 Decoding strategies and their failure modes

How the next token is picked from the distribution matters. Greedy decoding always takes the highest-probability token; sampling with temperature flattens or sharpens the distribution; top-k sampling restricts choices to the k most likely tokens. Holtzman et al. [2] showed that pure maximization strategies degenerate into repetitive, flat text, and made the key observation for this discussion: human text is not the highest-probability text. People routinely choose less predictable words and structures, and that variation is part of what makes human writing informative. Their fix, nucleus (top-p) sampling, truncates the distribution to the smallest set of tokens whose cumulative probability exceeds p [2]. Even with better decoding, though, the objective is unchanged: produce text that looks likely, not text that is true.

2.3 Why fluent falsehoods are a natural output

Because the model stores statistical patterns rather than a queryable database of facts, there is no internal distinction between recalling a fact and inventing one. When the prompt asks for something within the training distribution, likely continuations happen to be true often. When the prompt asks for something sparse, recent, or absent in the training data, the same machinery still

produces a fluent, confident continuation, now fabricated. Surveys of natural language generation identify exactly this as the mechanism behind hallucination [4]. The model has no way to flag the difference, because likelihood is the only signal it optimizes.

2.4 Structural differences from human writing

Human writing is planned and revised. A writer decides what to say, orders the points, drafts, and edits against an intended meaning. LLM generation has none of these stages, and the difference shows up in measurable properties of the text:

- Predictability: machine-generated text sits in a lower-perplexity region than human text. Human word choice is more surprising and idiosyncratic [2].
- Burstiness: human writing alternates short and long sentences and varies in complexity. Generated text is noticeably more uniform sentence to sentence.
- Register and stance: in a controlled comparison corpus, Guo et al. found ChatGPT answers to be longer, more objective, more formally structured, and less emotionally or personally marked than human expert answers to the same questions [3].
- Local coherence without global intent: an LLM maintains flow between adjacent sentences well, but has no communicative goal holding the whole answer together, which is why it can contradict itself across a long response or drift from the question.

These differences are also why detection tools that look only at surface statistics are unreliable, and why the safer engineering assumption is to verify content rather than judge style.

2.5 Implication for this project

If generation had an internal fact-checking stage, external verification would be redundant. It does not: the architecture described above produces text whose only guarantee is plausibility. A verification layer therefore has to be added from the outside, after generation, which is exactly the position this middleware occupies.

3. Datasets and Evaluation Benchmarks

3.1 Why FEVER and HaluEval alone are not enough

The initial plan used FEVER for fact-verification development and HaluEval for hallucination benchmarking. Both are useful baselines, but neither matches how this system actually works:

- FEVER (2018, 185,445 claims) was built by rewriting Wikipedia sentences, and its evidence is assumed to live inside Wikipedia [5]. It tests neither open-web retrieval nor retrieval against custom knowledge bases, and at eight years old it is saturated and likely contaminated in modern LLM pretraining data.
- HaluEval (2023, ~35,000 samples) builds its hallucinated samples synthetically, by ChatGPT sampling-then-filtering [6]. It therefore measures detection of machine-fabricated errors rather than naturally occurring hallucinations, and its authors acknowledged a length-bias issue; the HaluEval-Wild authors built a successor precisely because of these flaws [7].

Keeping both as baselines preserves comparability with prior work, but the primary evaluation needs datasets that exercise retrieval-in-the-loop verification and naturally occurring hallucinations.

3.2 Primary evaluation stack

The recommended stack is matched to the shape of the pipeline (decompose, retrieve, verify with NLI, score):

- RAGTruth (ACL 2024): about 18,000 RAG responses from multiple LLMs with word-level, human-annotated hallucination spans across QA, summarization, and data-to-text tasks [8]. It tests hallucination detection in exactly the RAG setting this middleware runs in, and supports span-level scoring.
- AVeriTeC (NeurIPS 2024): 4,568 real-world claims collected from 50 fact-checking organizations, with evidence retrieved from the open web, question-answer style evidence annotations, justifications, and temporal splits that block pretraining leakage [9]. The single best match for claim-level verification with retrieval. License note: CC-BY-NC, research use only.
- FActScore (EMNLP 2023): the de-facto atomic-fact methodology rather than a dataset. It decomposes a generation into atomic claims and scores the percentage supported by a knowledge source [10]. This validates the decomposition and support-classification stages end to end.
- LLM-AggreFact / MiniCheck (EMNLP 2024): aggregates 11 grounded-factuality datasets (RAGTruth included) into one benchmark and leaderboard for the question "is this claim supported by this document" [11]. It is the standard testbed for the NLI verification component.
- RAGChecker (NeurIPS 2024): a claim-level checking framework with separate retriever and generator diagnostic metrics, reporting the best correlation with human judgments among RAG metrics [12]. It tells us which stage of the pipeline is failing rather than only that something failed.

Proposed minimal stack: RAGTruth for detection, AVeriTeC for retrieval-based verification, FActScore-style scoring for the atomic-claim methodology, and LLM-AggreFact for the NLI component, with FEVER and HaluEval retained as baselines.

3.3 Additional datasets for breadth

- HaluEval-Wild (2024): real user queries filtered from ShareGPT, fixing the synthetic-sample problem of the original HaluEval [7].
- TruthfulQA (ACL 2022): 817 questions designed to bait common human misconceptions across 38 categories; a fast truthfulness smoke test [13].
- SimpleQA (OpenAI, 2024): 4,326 short fact-seeking questions with single, indisputable answers, adversarially collected; a clean short-form factuality measure [14].
- Domain options if the middleware is scoped to a vertical: SciFact (scientific claims with evidence abstracts), MedHallu (medical QA), FaithDial (knowledge-grounded dialogue).
- The Vectara Hallucination Leaderboard is an ongoing faithfulness ranking of LLMs; useful as a reference point, not as an evaluation dataset.

3.4 Summary table

Dataset	Venue / year	Size	Measures	Role
RAGTruth [8]	ACL 2024	~18,000 responses	Span-level hallucination detection in RAG outputs	Primary detection benchmark
AVeriTeC [9]	NeurIPS 2024	4,568 claims	Claim verification with open-web evidence	Primary retrieval-in-the-loop benchmark (research license)
FActScore [10]	EMNLP 2023	methodology	% of atomic claims supported by a source	Validates decomposition + scoring
LLM-AggreFact / MiniCheck [11]	EMNLP 2024	11 datasets	Grounded claim support classification	Testbed for the NLI component
RAGChecker [12]	NeurIPS 2024	framework	Claim-level RAG metrics, retriever vs generator	Stage-level diagnosis
HaluEval-Wild [7]	2024	real ShareGPT queries	Hallucination in real user-LLM interactions	Breadth: natural hallucinations
TruthfulQA [13]	ACL 2022	817 questions	Susceptibility to common misconceptions	Smoke test
SimpleQA [14]	2024	4,326 questions	Short-form factual accuracy	Breadth: factuality
FEVER [5]	NAACL 2018	185,445 claims	Fact verification against Wikipedia	Baseline only
HaluEval [6]	EMNLP 2023	~35,000 samples	Detection of synthetic hallucinations	Baseline only

4. Methods: System Architecture

4.1 Design overview

The middleware is a stateless proxy deployed between the client application and the LLM API. A request passes through unchanged to the LLM; the response is intercepted and run through a seven-stage verification pipeline before an enriched response is returned to the client. No component of the LLM is modified, and every stage is configurable: the LLM provider, the knowledge sources, the vector database, the NLI model, and the risk thresholds are all pluggable.

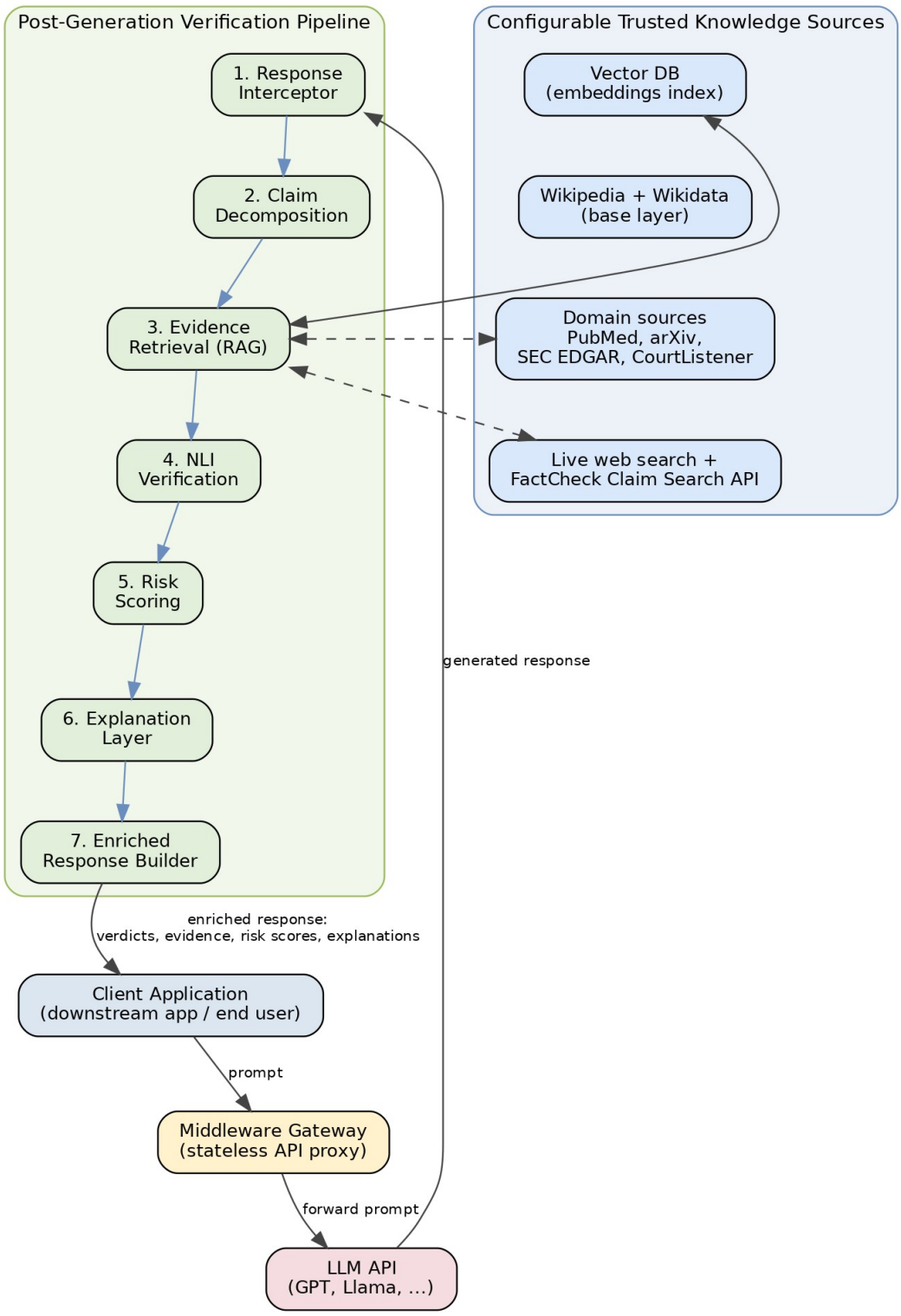


Figure 1: System context. The middleware gateway sits between the client and the LLM API; the verification pipeline (stages 1-7) draws on configurable trusted knowledge sources.

The end-to-end flow:

- 1. The client sends a prompt to the middleware gateway, which forwards it to the configured LLM API.
- 2. The response interceptor captures the generated text before it reaches the client.
- 3. Claim decomposition splits the response into atomic factual claims.
- 4. Evidence retrieval gathers supporting or contradicting passages for each claim from the configured trusted sources.
- 5. NLI verification classifies each claim against its evidence as supported, contradicted, or unsupported.
- 6. Risk scoring aggregates verdicts and confidences into claim-level and response-level hallucination scores.
- 7. The explanation layer attaches the evidence and reasoning behind flagged claims, and the enriched response builder returns everything to the client.

4.2 Verification pipeline detail

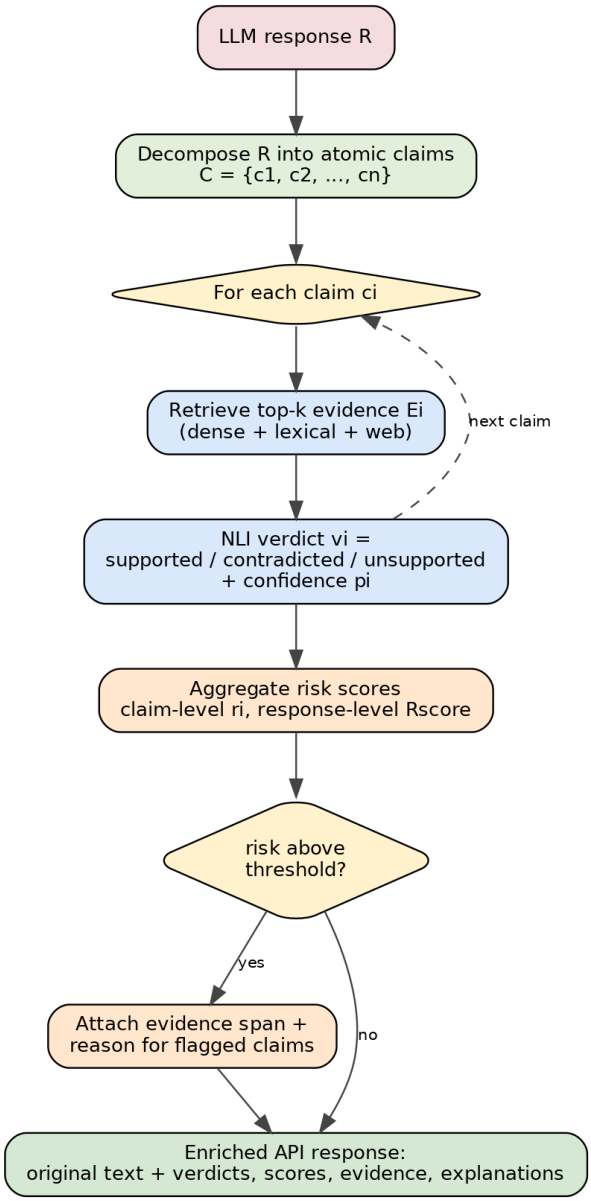


Figure 2: Verification pipeline data flow for one intercepted response R.

4.3 Stage 1-2: Interception and claim decomposition

The gateway forwards the prompt and captures the response synchronously, so verification adds latency but never changes the request semantics. The decomposition stage uses an LLM prompt or a fine-tuned extractor to break the response into atomic claims, following the FActScore methodology [10]: each claim must express exactly one verifiable fact, be self-contained (pronouns and references resolved), and preserve the original wording where possible. Opinions, greetings, and non-factual text are marked non-verifiable and skipped.

```
ALGORITHM 1: Claim decomposition
Input:  response R (plain text)
Output: list of atomic claims C = [c1, c2, ..., cn]

function DECOMPOSE(R):
    segments  <- split R into sentences
    claims    <- empty list
    for each sentence s in segments:
        atoms <- EXTRACT_ATOMIC_FACTS(s)   # LLM or fine-tuned extractor
        for each a in atoms:
            a <- RESOLVE_COREFERENCE(a, R) # make a self-contained
            if IS_VERIFIABLE(a):           # factual, not opinion
                claims.append(a)
    return claims
```

Algorithm 1: Atomic claim decomposition, following FActScore [10].

4.4 Stage 3: Evidence retrieval

Each claim is turned into one or more search queries, and evidence is gathered through three parallel channels: dense retrieval from an embedding index over the curated knowledge bases, lexical and API retrieval (BM25, live web search, and the Google FactCheck Claim Search API for claims that match prior fact checks), and domain-source adapters (for example PubMed or arXiv for scientific claims, SEC EDGAR for financial claims, CourtListener for legal claims). Results are merged with reciprocal rank fusion and reranked by a cross-encoder, yielding a small evidence set of passages, each carrying a source URL so the explanation layer can cite it.

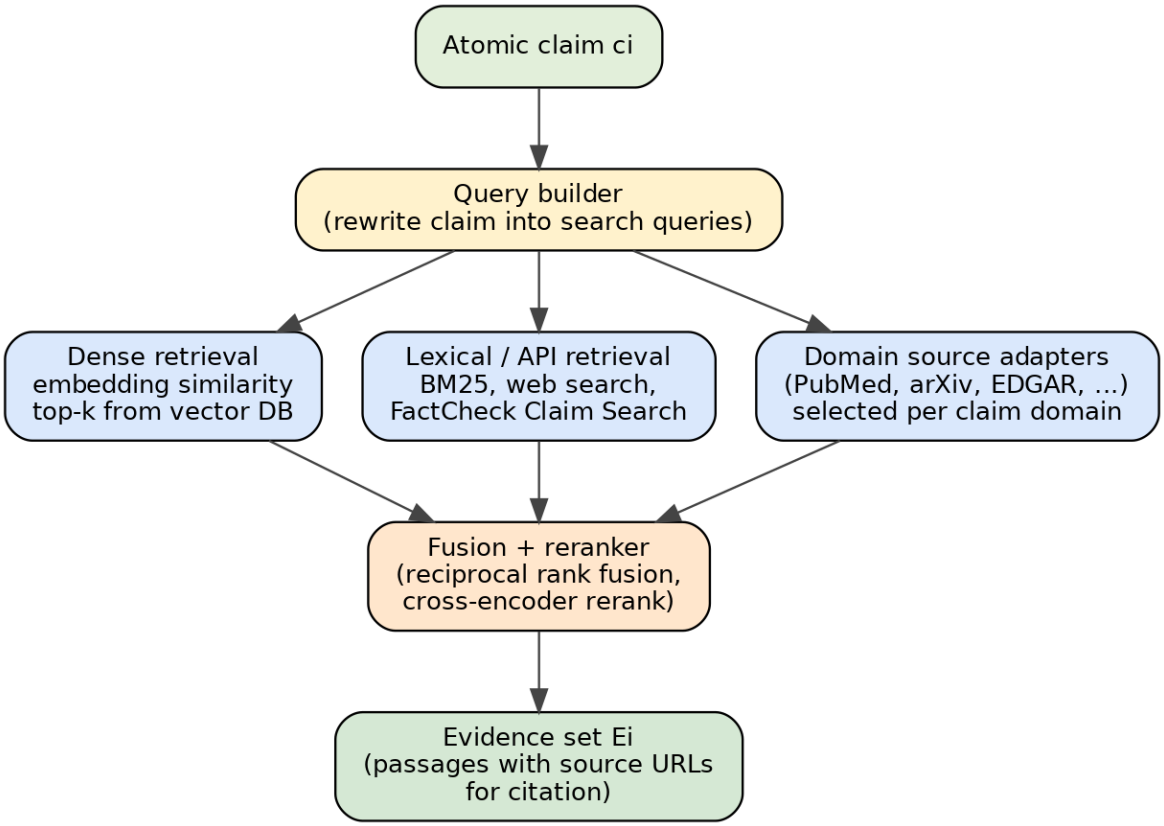


Figure 3: Evidence retrieval subsystem. Three retrieval channels are fused and reranked into a cited evidence set per claim.

```
ALGORITHM 2: Evidence retrieval for one claim
Input:  claim ci, sources S = {vector DB, web search, domain adapters}, top-k
Output: evidence set Ei (passages with source URLs)

function RETRIEVE(ci, S, k):
    queries <- BUILD_QUERIES(ci)            # 1-3 search queries
    results <- empty list
    for each q in queries:
        results += DENSE_SEARCH(q, S.vdb, k)     # embedding similarity
        results += LEXICAL_SEARCH(q, S.web, k)   # BM25 / web / FactCheck API
        results += DOMAIN_SEARCH(q, S.adapters)  # if claim domain matches
    fused   <- RECIPROCAL_RANK_FUSION(results)
    Ei      <- CROSS_ENCODER_RERANK(ci, fused)[:k]
    return Ei
```

4.5 Stage 4: NLI verification

Each claim-evidence pair is scored by a natural language inference model as entailment (supported), contradiction, or neutral (unsupported). MiniCheck-style models are the natural fit here because they are trained specifically for "is this claim supported by this document" judgments and are benchmarked on LLM-AggreFact [11]. A claim-level verdict combines the per-passage results: any confident contradiction overrides, otherwise the strongest entailment wins, and otherwise the claim is unsupported. Unsupported is a deliberate third class: it covers claims the trusted sources simply do not address, and it is scored and reported rather than silently passed.

```
ALGORITHM 3: NLI verification of one claim
Input:  claim ci, evidence set Ei, NLI model M, confidence threshold tau
Output: verdict vi in {SUPPORTED, CONTRADICTED, UNSUPPORTED}, confidence pi

function VERIFY(ci, Ei, M, tau):
    best_support <- 0; best_contradict <- 0
    for each passage e in Ei:
        (p_entail, p_contra, p_neutral) <- M(premise=e, hypothesis=ci)
        best_support    <- max(best_support, p_entail)
        best_contradict <- max(best_contradict, p_contra)
    if best_contradict >= tau:  return (CONTRADICTED, best_contradict)
    if best_support    >= tau:  return (SUPPORTED, best_support)
    return (UNSUPPORTED, max(best_support, best_contradict))
```

4.6 Stage 5: Risk scoring

Verdicts and confidences are aggregated into a claim-level risk score and a single response-level hallucination score. Contradicted claims carry the most weight because an actively wrong statement is worse than an unverifiable one; unsupported claims carry partial weight. The response-level score is a weighted fraction of claim risk over all verifiable claims, so a response is never punished for opinions or non-factual text. Thresholds divide responses into pass, review, and flag bands, and all thresholds are configuration, since different applications tolerate different risk levels.

```
ALGORITHM 4: Risk scoring
Input:  verdicts [(vi, pi)] for claims c1..cn, weights W, threshold theta
Output: claim risks [ri], response risk Rscore, decision

function SCORE(verdicts, W, theta):
    for each (vi, pi) in verdicts:
        if vi = CONTRADICTED:  ri <- W.contra * pi          # e.g. 1.0 * p
        if vi = UNSUPPORTED:   ri <- W.unsup  * (1 - pi)    # e.g. 0.5 * (1-p)
        if vi = SUPPORTED:     ri <- 0
    Rscore <- sum(ri) / count(verdicts)        # 0 .. W.contra
    decision <- PASS  if Rscore < theta.low
             <- REVIEW if theta.low <= Rscore < theta.high
             <- FLAG  if Rscore >= theta.high
    return ([ri], Rscore, decision)
```

4.7 Stage 6: Explanation layer

Every flagged claim carries an explanation: the claim text, its verdict, the top evidence passages with source URLs, and a short natural-language reason (for example, which evidence sentence contradicts the claim and what it says instead). This is what separates the system from a bare confidence score, and it is the main reason the knowledge sources must be citable (Section 5): the user has to be able to open the evidence and check it.

4.8 Stage 7: Enriched API response

The client receives the original response unchanged, plus verification metadata it can render, log, or act on. A downstream application might highlight flagged spans, show evidence on hover, or block responses above a risk threshold.

```
{
```

```
  "response": "<original LLM output, unchanged>",
  "verification": {
    "response_risk_score": 0.31,
    "decision": "REVIEW",
    "claims": [
      {
        "claim": "The population of X is 2.1 million.",
        "verdict": "contradicted",
        "confidence": 0.92,
        "claim_risk": 0.92,
        "evidence": [
          { "source": "https://en.wikipedia.org/wiki/X",
            "passage": "As of the 2021 census, X had a
            population of 1.4 million." }
        ],
        "explanation": "Wikipedia (2021 census) cites 1.4
        million, contradicting the claim.",
      }
    ]
  }
}
```

Example enriched response object returned to the client.

4.9 Overall pipeline

```
ALGORITHM 0: End-to-end middleware pipeline
Input:  client prompt q, config {LLM, sources S, NLI model M, thresholds}
Output: enriched response object

function MIDDLEWARE(q, config):
    R        <- config.LLM.generate(q)          # black-box call
    C        <- DECOMPOSE(R)                    # Algorithm 1
    verdicts <- empty list
    for each ci in C:
        Ei <- RETRIEVE(ci, config.S, k)         # Algorithm 2
        vi <- VERIFY(ci, Ei, config.M, tau)     # Algorithm 3
        verdicts.append(vi with evidence Ei)
    (risks, Rscore, decision) <- SCORE(verdicts, W, theta)  # Algorithm 4
    explanations <- EXPLAIN(flagged claims, evidence)
    return BUILD_ENRICHED_RESPONSE(R, C, verdicts, risks,
                                   Rscore, decision, explanations)
```

5. Knowledge Base Design

5.1 Why a pretraining corpus is not a verification KB

One early idea was to use a single large, ready-made corpus as the knowledge base across all domains, taking IBM Granite's published training mixture (arXiv, PubMed Central, SEC filings, patents, GitHub, Stack Exchange, Common Crawl, Wikipedia, and others, as described in Red Hat's breakdown of the Granite training data) as the candidate. That does not work, for four reasons:

- It is not a product. IBM published the composition and lineage of the training mixture for transparency; there is no packaged, licensed corpus available to download and index, and some components (such as Webhose) are proprietary feeds.
- Category mismatch. A pretraining corpus is a stale, unfiltered snapshot optimized for token volume. A verification knowledge base must be current, curated for trustworthiness, and citable, because the explanation layer has to show users the evidence. Hacker News threads and raw web crawl cannot serve as trusted evidence.
- Baked-in staleness. Components of that mixture end years ago (for example Hacker News content ends in 2018 and SEC filings in 2022), while verification needs sources that stay current.
- Circularity risk. Checking an LLM's output against the same kind of web data LLMs trained on can confirm shared errors instead of catching them. Independent, curated sources are what make verification meaningful.

5.2 Curated layered sources instead

The alternative is not one mega-corpus but a set of curated per-domain trusted sources, which is exactly the plug-in architecture the middleware already proposes:

- Base layer: Wikipedia dump plus Wikidata. Free, citable, regularly updated, and the same convention FEVER and FActScore use, so evaluation and deployment share an evidence convention [5][10].
- Domain layers, pulled directly from the authoritative source rather than from a training-data snapshot: PubMed and arXiv for science, SEC EDGAR for finance, CourtListener for legal.
- Freshness layer: live web search and the Google FactCheck Claim Search API for recent or breaking claims, mirroring how AVeriTeC sourced its evidence [9].

Each source ships as an adapter behind the retrieval interface in Figure 3, so adding a source is a configuration change, not a code change.

6. Conclusion

LLMs generate text by repeatedly picking the most plausible next token, with no planning stage and no truth check, which is why fluent hallucinations are a structural property of the technology rather than an occasional bug. This project adds the missing check from the outside: a post-generation middleware that decomposes responses into atomic claims, retrieves evidence from configurable trusted sources, verifies each claim with NLI, and returns risk scores and human-readable explanations alongside the original output. The design is model-agnostic and source-agnostic, and its evaluation plan is grounded in current benchmarks (RAGTruth, AVeriTeC, FActScore, LLM-AggreFact, RAGChecker) that exercise the same pipeline stages the system implements, with FEVER and HaluEval kept as historical baselines.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," NeurIPS, 2017. https://arxiv.org/abs/1706.03762

[2] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The Curious Case of Neural Text Degeneration," ICLR, 2020. https://arxiv.org/abs/1904.09751

[3] B. Guo, X. Zhang, Z. Wang, M. Jiang, J. Nie, Y. Ding, J. Yue, and Y. Wu, "How Close is ChatGPT to Human Experts? Comparison Corpus, Evaluation, and Detection," arXiv:2301.07597, 2023. https://arxiv.org/abs/2301.07597

[4] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of Hallucination in Natural Language Generation," ACM Computing Surveys, 2023. https://arxiv.org/abs/2202.03629

[5] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, "FEVER: a Large-scale Dataset for Fact Extraction and VERification," NAACL, 2018. https://aclanthology.org/N18-1074/

[6] J. Li, X. Cheng, W. X. Zhao, J.-Y. Nie, and J.-R. Wen, "HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models," EMNLP, 2023. https://aclanthology.org/2023.emnlp-main.397/

[7] Z. Zhu, Y. Yang, and Z. Sun, "HaluEval-Wild: Evaluating Hallucinations of Language Models in the Wild," arXiv:2403.04307, 2024. https://arxiv.org/abs/2403.04307

[8] C. Niu, Y. Wu, J. Zhu, S. Xu, K. Shum, R. Zhong, J. Song, and T. Zhang, "RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models," ACL, 2024. https://aclanthology.org/2024.acl-long.585/

[9] M. Schlichtkrull, Z. Guo, and A. Vlachos, "AVeriTeC: A Dataset for Real-world Claim Verification with Evidence from the Web," NeurIPS, 2024. https://github.com/MichSchli/AVeriTeC

[10] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation," EMNLP, 2023. https://aclanthology.org/2023.emnlp-main.741/

[11] L. Tang, P. Laban, and G. Durrett, "MiniCheck: Efficient Fact-Checking of LLMs on Grounding Documents," EMNLP, 2024. https://llm-aggrefact.github.io/

[12] D. Ru, L. Qiu, X. Hu, T. Zhang, P. Shi, S. Chang, et al., "RAGChecker: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation," NeurIPS, 2024. https://arxiv.org/abs/2408.08067

[13] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring How Models Mimic Human Falsehoods," ACL, 2022. https://aclanthology.org/2022.acl-long.229/

[14] J. Wei, N. Sun, et al., "Measuring Short-Form Factuality in Large Language Models (SimpleQA)," arXiv:2411.04368, 2024. https://arxiv.org/abs/2411.04368